Complete Notes on Vectors

Machine Learning, Deep Learning & AI

Contents

1 Introduction to Vectors **3**

1.1 Why Vectors Matter in ML/DL/AI 3

1.1.1 Why Are Vectors Fundamental in ML/DL/AI? 3

1.2 Notation and Conventions 7

1.2.1 Mathematical Notation 7

1.2.2 Special Vector Types 7

2 Vector Representations **9**

2.1 Column Vectors vs Row Vectors 9

2.1.1 Transpose Operation 9

2.1.2 Why Shape Matters in ML/DL 9

2.2 Special Vectors 12

2.2.1 Standard Basis Vectors 12

2.2.2 Unit Vectors (Normalized Vectors) 13

3 Vector Magnitude and Distance **15**

3.1 Vector Norms (Magnitudes) 15

3.1.1 L2 Norm (Euclidean Norm) 15

3.1.2 L1 Norm (Manhattan Norm) 16

3.1.3 L∞ Norm (Max Norm) 17

3.2 Distance Between Vectors 19

4 Vector Operations **22**

4.1 Scalar-Vector Operations 22

4.1.1 Scalar Addition and Subtraction 22

4.1.2 Scalar Multiplication and Division 23

4.2 Vector-Vector Operations 25

4.2.1 Element-wise Addition and Subtraction 25

4.3 Dot Product (Inner Product) 26

5 Summary and Key Formulas **31**

6 Linear Independence and Basis **31**

6.1 Linear Combinations 31

6.2 Linear Independence 33

6.3 Span 35

6.4 Basis 37

7 Vector Spaces **39**

7.1 Subspaces 39

7.2 Hyperplanes and Decision Boundaries 41

8 Advanced ML/DL Applications **45**

8.1 Embedding Spaces 46

8.2 Attention Mechanisms 49

8.3 Gradient Descent Optimization 51

9 Assessment Questions **53**

9.1 Basic Level (11-20) 53

9.2 Intermediate Level (21-30) 53

9.3 Advanced Level (31-40) 53

9.4 Solutions to Selected Questions 54

10 Summary and Key Takeaways **56**

1 Introduction to Vectors

1.1 Why Vectors Matter in ML/DL/AI

What is a Vector?

A **vector** is an ordered collection of numbers (called components or elements) that represents a quantity with both **magnitude** (size/length) and **direction**.

Mathematical Representation:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n$$

where:

- $\mathbf{v}$ is the vector (bold lowercase letter)
- $v_1, v_2, \ldots, v_n$ are the components/elements
- n is the dimension of the vector
- $\mathbb{R}^n$ is the n-dimensional real space

1.1.1 Why Are Vectors Fundamental in ML/DL/AI?

1. Data Representation:

Every piece of data in machine learning is represented as a vector. This numerical representation allows computers to process, analyze, and learn from data.

Example 1.1: Customer Data as Vector

Scenario: A bank wants to predict loan default risk.
Customer Profile:

- Age: 35 years
- Annual Income: $65,000
- Credit Score: 720
- Loan Amount: $250,000
- Employment Years: 8

Vector Representation:

$$\mathbf{x}_{\text{customer}} = \begin{bmatrix} 35 \\ 65000 \\ 720 \\ 250000 \\ 8 \end{bmatrix} \in \mathbb{R}^5$$

This is a 5-dimensional feature vector. The ML model will process this vector through mathematical operations (dot products, matrix multiplications) to make predictions.
Why This Matters:

- Enables batch processing of millions of customers
- Allows mathematical operations: distances, similarities
- Can be fed into neural networks

2. Efficient Computation:

Vectors enable parallel processing and GPU acceleration, which is crucial for training large neural networks.

Example 1.2: Image Representation

Scenario: Processing a grayscale image for digit recognition (MNIST dataset).
Original Image: 28×28 pixels
Vector Representation: Flatten the 28×28 matrix into a single vector:

$$\mathbf{x}_{\text{image}} = [x_1, x_2, x_3, \ldots, x_{784}]^T \in \mathbb{R}^{784}$$

Each x_i represents pixel intensity (0-255).
Example for digit "5":

$$\mathbf{x}_{\text{image}} = \begin{bmatrix} 0 \\ 0 \\ 245 \\ 238 \\ \vdots \\ 12 \\ 0 \end{bmatrix}$$

Why Vectorization Matters:

- Can process entire batch of images simultaneously
- Matrix operations on GPU are 100× faster than loops
- Neural network layers expect vector inputs

3. Feature Engineering:

Vectors encode complex relationships and patterns in data.

Example 1.3: House Price Prediction

Dataset: Real estate data for predicting house prices.
Raw Data:

- Location: "Downtown"
- House Type: "Apartment"
- Square Feet: 1200
- Bedrooms: 2
- Age: 15 years

Feature Vector (after encoding):

$$\mathbf{x}_{\text{house}} = \begin{bmatrix} 1200 \\ 2 \\ 15 \\ 1 \\ 0 \\ 0.85 \end{bmatrix}$$

where: $x_1 = 1200$ (square feet), $x_2 = 2$ (bedrooms), $x_3 = 15$ (age), $x_4 = 1$ (apartment $= 1$, house $= 0$), $x_5 = 0$ (suburban $= 1$, downtown $= 0$), $x_6 = 0.85$ (location quality score)
Linear Regression Model:

$$\text{Price} = \mathbf{w}^T \mathbf{x}_{\text{house}} + b$$

With weights: $\mathbf{w} = [150, 50000, -2000, -30000, 20000, 100000]^T$, $b = 50000$
Calculation:

$$\begin{aligned} \text{Price} &= (150)(1200) + (50000)(2) + (-2000)(15) \\ &\quad + (-30000)(1) + (20000)(0) + (100000)(0.85) + 50000 \\ &= 180000 + 100000 - 30000 - 30000 + 0 + 85000 + 50000 \\ &= \$355,000 \end{aligned}$$

This shows how vectors enable prediction through simple arithmetic!

4. Model Parameters:

Neural network weights, biases, and gradients are all vectors.

Example 1.4: Single Neuron in Neural Network

Scenario: A neuron that decides "Will customer buy premium subscription?"
Inputs (user features):

$$\mathbf{x} = \begin{bmatrix} 25 \\ 3 \\ 12 \end{bmatrix} = \begin{bmatrix} \text{age} \\ \text{years as customer} \\ \text{monthly logins} \end{bmatrix}$$

Learned Weights:

$$\mathbf{w} = \begin{bmatrix} 0.02 \\ 0.15 \\ 0.08 \end{bmatrix}$$

Bias: $b = -1.5$
Neuron Computation:

$$\begin{aligned} z &= \mathbf{w}^T\mathbf{x} + b \\ &= (0.02)(25) + (0.15)(3) + (0.08)(12) + (-1.5) \\ &= 0.5 + 0.45 + 0.96 - 1.5 \\ &= 0.41 \end{aligned}$$

Activation (Sigmoid):

$$a = \sigma(z) = \frac{1}{1 + e^{-0.41}} = \frac{1}{1 + 0.664} = 0.601$$

Interpretation: 60.1% probability of buying premium subscription.
This is the fundamental operation in every neural network!

1.2 Notation and Conventions

1.2.1 Mathematical Notation

Vector Notation Standards

1. **Bold Lowercase Letters:** Vectors are denoted by bold lowercase letters.

$$\mathbf{v}, \mathbf{w}, \mathbf{x}, \mathbf{y}, \mathbf{a}, \mathbf{b}$$

2. **Component Notation:** Individual elements use subscripts.

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

3. **Dimension:** The number of components is the dimension.

$$\mathbf{v} \in \mathbb{R}^n \text{ means } \mathbf{v} \text{ has } n \text{ components}$$

4. **Indexing:**
   - Mathematics: 1-indexed $(v_1, v_2, \ldots, v_n)$
   - Programming: 0-indexed (v[0], v[1], ..., v[n-1])

1.2.2 Special Vector Types

Important Vector Types

1. **Zero Vector:** All components are zero.

$$\mathbf{0} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

Properties: Additive identity: $\mathbf{v} + \mathbf{0} = \mathbf{v}$, No direction (magnitude $= 0$)

2. **Unit Vector:** Magnitude equals 1. $\|\hat{\mathbf{v}}\| = 1$
3. **Sparse Vector:** Most components are zero.

$$\mathbf{s} = [0, 0, 5, 0, 0, 0, 3, 0, \ldots, 0]$$

4. **Dense Vector:** Few or no zero components.

$$\mathbf{d} = [0.23, -0.45, 0.67, 0.89, -0.12, \ldots]$$

Example 1.5: Sparse vs Dense Vectors in NLP

Scenario: Representing the sentence "I love machine learning"
Vocabulary: {I, love, hate, machine, learning, deep, neural, network}
Method 1: Sparse - Bag of Words (One-Hot Encoding)
Word "machine" represented as:

$$\mathbf{v}_{\text{machine}} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \in \mathbb{R}^8$$

Properties:

- Only 1 non-zero element
- High dimensionality (vocabulary size = 10,000+)
- No semantic meaning captured

Method 2: Dense - Word Embeddings (Word2Vec)
Word "machine" represented as:

$$\mathbf{e}_{\text{machine}} = \begin{bmatrix} 0.23 \\ -0.45 \\ 0.67 \\ 0.12 \\ -0.89 \end{bmatrix} \in \mathbb{R}^5$$

Properties:

- All elements are non-zero
- Lower dimensionality (50-300 dimensions)
- Captures semantic relationships

Semantic Relationship Example:

$$\mathbf{e}_{\text{machine}} - \mathbf{e}_{\text{computer}} \approx \mathbf{e}_{\text{engine}} - \mathbf{e}_{\text{motor}}$$

Why Dense is Better:

- Memory efficient: 300D vs 100,000D
- Captures meaning: similar words have similar vectors
- Better for neural networks: smoother gradients

2 Vector Representations

2.1 Column Vectors vs Row Vectors

Column Vector

A **column vector** is an $n \times 1$ matrix (n rows, 1 column).
Standard Form:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_n \end{bmatrix} \quad \text{(shape: } n \times 1\text{)}$$

This is the default representation in mathematics and ML.

Row Vector

A **row vector** is a $1 \times n$ matrix (1 row, n columns).
Standard Form:

$$\mathbf{v}^T = [v_1, v_2, v_3, \ldots, v_n] \quad \text{(shape: } 1 \times n\text{)}$$

Note: The superscript T denotes transpose.

2.1.1 Transpose Operation

Vector Transpose

The **transpose** operation converts row vectors to column vectors and vice versa.
Notation: $\mathbf{v}^T$ or $\mathbf{v}^\top$
Operation:

$$\text{If } \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \end{bmatrix}, \quad \text{then } \mathbf{v}^T = [v_1, v_2, v_3]$$

Properties:

- $(\mathbf{v}^T)^T = \mathbf{v}$
- $(k\mathbf{v})^T = k\mathbf{v}^T$
- $(\mathbf{u} + \mathbf{v})^T = \mathbf{u}^T + \mathbf{v}^T$

2.1.2 Why Shape Matters in ML/DL

Critical Importance of Vector Shape

In deep learning frameworks (PyTorch, TensorFlow), **shape mismatches** are the most common error!
Matrix Multiplication Rules:

$$(m \times n) \cdot (n \times p) = (m \times p) \quad \checkmark$$

$$(m \times n) \cdot (p \times q) = \text{ERROR if } n \neq p \quad \times$$

Example 2.1: Shape Errors in Neural Networks

Scenario: Building a simple neural network layer.
Layer Configuration:

- Input features: 4
- Hidden neurons: 3
- Batch size: 1 (single sample)

Input Vector:

$$\mathbf{x} = \begin{bmatrix} 1.0 \\ 2.0 \\ 3.0 \\ 4.0 \end{bmatrix} \in \mathbb{R}^{4\times 1}$$

Weight Matrix:

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 1.0 & 1.1 & 1.2 \end{bmatrix} \in \mathbb{R}^{3\times 4}$$

Bias Vector:

$$\mathbf{b} = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} \in \mathbb{R}^{3\times 1}$$

Forward Pass Calculation: $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$
Step-by-Step:

$$\mathbf{W}\mathbf{x} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 1.0 & 1.1 & 1.2 \end{bmatrix} \begin{bmatrix} 1.0 \\ 2.0 \\ 3.0 \\ 4.0 \end{bmatrix}$$

$$= \begin{bmatrix} (0.1)(1.0) + (0.2)(2.0) + (0.3)(3.0) + (0.4)(4.0) \\ (0.5)(1.0) + (0.6)(2.0) + (0.7)(3.0) + (0.8)(4.0) \\ (0.9)(1.0) + (1.0)(2.0) + (1.1)(3.0) + (1.2)(4.0) \end{bmatrix}$$

$$= \begin{bmatrix} 0.1 + 0.4 + 0.9 + 1.6 \\ 0.5 + 1.2 + 2.1 + 3.2 \\ 0.9 + 2.0 + 3.3 + 4.8 \end{bmatrix} = \begin{bmatrix} 3.0 \\ 7.0 \\ 11.0 \end{bmatrix}$$

Add Bias:

$$\mathbf{z} = \begin{bmatrix} 3.0 \\ 7.0 \\ 11.0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 3.1 \\ 7.2 \\ 11.3 \end{bmatrix}$$

Shape Check: $(3 \times 4) \cdot (4 \times 1) + (3 \times 1) = (3 \times 1)$ ✓
Common Error: If we mistakenly use $\mathbf{x}^T$ (row vector): $(3 \times 4) \cdot (1 \times 4) =$ ERROR! ×
Dimensions don't match! This crashes the program.

Example 2.2: Batch Processing

Scenario: Processing multiple samples simultaneously.

Batch of 3 samples:

$$\mathbf{X} = \begin{bmatrix} 1.0 & 2.0 & 3.0 & 4.0 \\ 0.5 & 1.5 & 2.5 & 3.5 \\ 2.0 & 3.0 & 4.0 & 5.0 \end{bmatrix} \in \mathbb{R}^{3\times 4}$$

Each row is one sample (one customer, one image, etc.)

Same Weight Matrix:

$$\mathbf{W} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.8 \\ 0.9 & 1.0 & 1.1 & 1.2 \end{bmatrix} \in \mathbb{R}^{3\times 4}$$

Batch Forward Pass: $\mathbf{Z} = \mathbf{X}\mathbf{W}^T + \mathbf{b}$

Calculation:

$$\mathbf{X}\mathbf{W}^T = \begin{bmatrix} 1.0 & 2.0 & 3.0 & 4.0 \\ 0.5 & 1.5 & 2.5 & 3.5 \\ 2.0 & 3.0 & 4.0 & 5.0 \end{bmatrix} \begin{bmatrix} 0.1 & 0.5 & 0.9 \\ 0.2 & 0.6 & 1.0 \\ 0.3 & 0.7 & 1.1 \\ 0.4 & 0.8 & 1.2 \end{bmatrix}$$

$$= \begin{bmatrix} 3.0 & 7.0 & 11.0 \\ 2.6 & 6.1 & 9.6 \\ 4.0 & 9.4 & 14.8 \end{bmatrix}$$

Add Bias (broadcasts to all rows):

$$\mathbf{Z} = \begin{bmatrix} 3.0 & 7.0 & 11.0 \\ 2.6 & 6.1 & 9.6 \\ 4.0 & 9.4 & 14.8 \end{bmatrix} + \begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} = \begin{bmatrix} 3.1 & 7.2 & 11.3 \\ 2.7 & 6.3 & 9.9 \\ 4.1 & 9.6 & 15.1 \end{bmatrix}$$

Result: 3 samples processed simultaneously! This is 100× faster on GPU.

2.2 Special Vectors

2.2.1 Standard Basis Vectors

Standard Basis Vectors

In $\mathbb{R}^n$, the **standard basis vectors** are:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \ldots, \quad \mathbf{e}_n = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

Properties:

- Each has exactly one 1 and rest 0s
- Mutually orthogonal: $\mathbf{e}_i \cdot \mathbf{e}_j = 0$ if $i \neq j$
- Unit length: $\|\mathbf{e}_i\| = 1$
- Span entire space: any vector is their linear combination

Example 2.3: One-Hot Encoding (Categorical Variables)

Scenario: Encoding the color of a car for ML model.
Categories: {Red, Blue, Green, Black, White}
One-Hot Encoding:

$$\text{Red} \to \mathbf{e}_1 = \begin{bmatrix}1\\0\\0\\0\\0\end{bmatrix}, \quad \text{Blue} \to \mathbf{e}_2 = \begin{bmatrix}0\\1\\0\\0\\0\end{bmatrix}, \quad \text{Green} \to \mathbf{e}_3 = \begin{bmatrix}0\\0\\1\\0\\0\end{bmatrix}$$

$$\text{Black} \to \mathbf{e}_4 = \begin{bmatrix}0\\0\\0\\1\\0\end{bmatrix}, \quad \text{White} \to \mathbf{e}_5 = \begin{bmatrix}0\\0\\0\\0\\1\end{bmatrix}$$

Full Feature Vector for a Red Car:

$$\mathbf{x}_{\text{car}} = \begin{bmatrix}\text{price}\\\text{mileage}\\\text{Red}\\\text{Blue}\\\text{Green}\\\text{Black}\\\text{White}\end{bmatrix} = \begin{bmatrix}25000\\50000\\1\\0\\0\\0\\0\end{bmatrix}$$

Why Use One-Hot?

- Cannot use numbers: "Red=1, Blue=2" implies ordering
- Model would think Blue is "twice" Red—meaningless!
- One-hot treats all categories equally

2.2.2 Unit Vectors (Normalized Vectors)

Unit Vector

A **unit vector** $\hat{\mathbf{v}}$ has magnitude exactly equal to 1.
Formula:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

where $\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$ is the magnitude.
Verification:

$$\|\hat{\mathbf{v}}\| = \left\|\frac{\mathbf{v}}{\|\mathbf{v}\|}\right\| = \frac{\|\mathbf{v}\|}{\|\mathbf{v}\|} = 1 \quad \checkmark$$

Example 2.4: Computing Unit Vector

Given: $\mathbf{v} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$

Step 1: Calculate magnitude

$$\|\mathbf{v}\| = \sqrt{v_1^2 + v_2^2} = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5$$

Step 2: Divide by magnitude

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|} = \frac{1}{5}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}$$

Step 3: Verify

$$\|\hat{\mathbf{v}}\| = \sqrt{(0.6)^2 + (0.8)^2} = \sqrt{0.36 + 0.64} = \sqrt{1.0} = 1.0 \quad \checkmark$$

Example 2.5: Feature Normalization in ML

Scenario: Normalizing features for gradient descent convergence.

Original Feature Vector:

$$\mathbf{x}_{\text{raw}} = \begin{bmatrix} 5000 \text{ (income)} \\ 2 \text{ (bedrooms)} \\ 30 \text{ (age)} \end{bmatrix}$$

Problem: Income dominates! Gradients will be unstable.

Solution: L2 Normalization

Step 1: Calculate L2 norm

$$\|\mathbf{x}_{\text{raw}}\| = \sqrt{5000^2 + 2^2 + 30^2} = \sqrt{25000000 + 4 + 900} = \sqrt{25000904} \approx 5000.09$$

Step 2: Normalize

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x}_{\text{raw}}}{\|\mathbf{x}_{\text{raw}}\|} = \frac{1}{5000.09}\begin{bmatrix} 5000 \\ 2 \\ 30 \end{bmatrix} \approx \begin{bmatrix} 0.9998 \\ 0.0004 \\ 0.0060 \end{bmatrix}$$

Now all features are in range [0, 1]!

Benefits:

- Gradient descent converges faster
- No feature dominates learning
- Neural networks train more stably
- Required for cosine similarity

3 Vector Magnitude and Distance

3.1 Vector Norms (Magnitudes)

Vector Norm (General)

A **norm** is a function that assigns a positive length or size to a vector.
Properties: For any norm $\|\cdot\|$:

1. **Non-negativity:** $\|\mathbf{v}\| \geq 0$, and $\|\mathbf{v}\| = 0$ iff $\mathbf{v} = \mathbf{0}$
2. **Scalar multiplication:** $\|k\mathbf{v}\| = |k|\|\mathbf{v}\|$
3. **Triangle inequality:** $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$

3.1.1 L2 Norm (Euclidean Norm)

L2 Norm

The **L2 norm** (Euclidean norm) measures the straight-line distance from origin.
Formula:

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2} = \sqrt{\sum_{i=1}^{n} v_i^2}$$

Alternative notation: $\|\mathbf{v}\|$ (when subscript omitted, L2 is assumed)
Geometric meaning: Pythagorean theorem in n dimensions!

Example 3.1: L2 Norm Calculation

Given: $\mathbf{v} = \begin{bmatrix} 3 \\ -4 \\ 12 \end{bmatrix}$
Calculate L2 norm:

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + v_3^2} = \sqrt{3^2 + (-4)^2 + 12^2} = \sqrt{9 + 16 + 144} = \sqrt{169} = 13$$

Interpretation: The vector is 13 units away from the origin.

Example 3.2: Regularization in Ridge Regression

Scenario: Preventing overfitting by penalizing large weights.
Linear Model: $y = \mathbf{w}^T\mathbf{x} + b$
Loss Function (without regularization):

$$L_{\text{MSE}} = \frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \hat{y}^{(i)})^2$$

Ridge Regression (L2 Regularization):

$$L_{\text{Ridge}} = \frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \hat{y}^{(i)})^2 + \lambda\|\mathbf{w}\|_2^2$$

Concrete Example:

Learned weights: $\mathbf{w} = \begin{bmatrix} 2.5 \\ -1.8 \\ 3.2 \\ 0.9 \end{bmatrix}$, Regularization parameter: $\lambda = 0.1$

Calculate penalty:

$$\begin{aligned}\|\mathbf{w}\|_2^2 &= w_1^2 + w_2^2 + w_3^2 + w_4^2 = (2.5)^2 + (-1.8)^2 + (3.2)^2 + (0.9)^2 \\ &= 6.25 + 3.24 + 10.24 + 0.81 = 20.54\end{aligned}$$

Regularization term: $\lambda\|\mathbf{w}\|_2^2 = 0.1 \times 20.54 = 2.054$
Total loss: $L_{\text{Ridge}} = L_{\text{MSE}} + 2.054$
Effect: During training, gradient descent will shrink large weights to reduce this penalty, preventing overfitting!

3.1.2 L1 Norm (Manhattan Norm)

L1 Norm

The **L1 norm** (Manhattan norm, taxicab norm) sums absolute values.
Formula:

$$\|\mathbf{v}\|_1 = |v_1| + |v_2| + \cdots + |v_n| = \sum_{i=1}^{n}|v_i|$$

Geometric meaning: Distance traveling along axis-aligned paths (like city blocks).

Example 3.3: L1 vs L2 Norm

Given: $\mathbf{v} = \begin{bmatrix} 3 \\ -4 \\ 2 \end{bmatrix}$
L1 Norm: $\|\mathbf{v}\|_1 = |3| + |-4| + |2| = 3 + 4 + 2 = 9$
L2 Norm: $\|\mathbf{v}\|_2 = \sqrt{3^2 + (-4)^2 + 2^2} = \sqrt{9 + 16 + 4} = \sqrt{29} \approx 5.385$
Key Observation: $\|\mathbf{v}\|_1 \geq \|\mathbf{v}\|_2$ always! L1 is "longer" because you can't cut corners—must travel along axes.

Example 3.4: Lasso Regression (L1 Regularization)

Scenario: Feature selection through sparsity.

Lasso Loss:

$$L_{\text{Lasso}} = \frac{1}{m}\sum_{i=1}^{m}(y^{(i)} - \hat{y}^{(i)})^2 + \lambda\|\mathbf{w}\|_1$$

Learned weights: $\mathbf{w} = \begin{bmatrix} 2.5 \\ -0.02 \\ 3.2 \\ 0.01 \end{bmatrix}$

Calculate L1 penalty:

$$\|\mathbf{w}\|_1 = |2.5| + |-0.02| + |3.2| + |0.01| = 2.5 + 0.02 + 3.2 + 0.01 = 5.73$$

With $\lambda = 0.1$: penalty $= 0.1 \times 5.73 = 0.573$

Magic of L1: During training, small weights (w_2, w_4) get pushed to **exactly zero**!

After Lasso: $\mathbf{w} = \begin{bmatrix} 2.5 \\ 0 \\ 3.2 \\ 0 \end{bmatrix}$

Interpretation: Features 2 and 4 are unimportant—automatic feature selection!

Why L1 creates sparsity:

- L1 has "sharp corners" at axes
- Gradients are constant (not proportional to weight size)
- Small weights easily hit zero

Comparison:

Property	L1 (Lasso)	L2 (Ridge)
Sparsity	Yes (many zeros)	No
Feature selection	Yes	No
Solution	May not be unique	Unique
Robust to outliers	Yes	No

3.1.3 L∞ Norm (Max Norm)

L∞ Norm

The **L∞ norm** (maximum norm, Chebyshev norm) takes the largest absolute value.

Formula:

$$\|\mathbf{v}\|_\infty = \max(|v_1|, |v_2|, \ldots, |v_n|) = \max_{1 \le i \le n} |v_i|$$

Example 3.5: L∞ Norm Calculation

Given: $\mathbf{v} = \begin{bmatrix} 3 \\ -8 \\ 2 \\ 5 \end{bmatrix}$

Calculate:

$$\|\mathbf{v}\|_\infty = \max(|3|, |-8|, |2|, |5|) = \max(3, 8, 2, 5) = 8$$

Example 3.6: Gradient Clipping

Scenario: Preventing exploding gradients in RNNs.
Gradient vector:

$$\nabla L = \begin{bmatrix} 0.5 \\ -12.3 \\ 2.1 \\ 0.8 \end{bmatrix}$$

Problem: Large gradient component (-12.3) will cause huge weight update!
Solution: Clip using L∞ norm
Maximum allowed: clip_value = 5.0
Step 1: Check max component

$$\|\nabla L\|_\infty = \max(|0.5|, |-12.3|, |2.1|, |0.8|) = 12.3$$

Step 2: Clip if exceeds threshold

$$\text{scaling_factor} = \frac{\text{clip_value}}{\|\nabla L\|_\infty} = \frac{5.0}{12.3} \approx 0.407$$

$$\nabla L_{\text{clipped}} = 0.407 \times \begin{bmatrix} 0.5 \\ -12.3 \\ 2.1 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 0.203 \\ -5.0 \\ 0.855 \\ 0.326 \end{bmatrix}$$

Verification: $\|\nabla L_{\text{clipped}}\|_\infty = \max(0.203, 5.0, 0.855, 0.326) = 5.0$ ✓
Now safe to use! Network won't diverge.

3.2 Distance Between Vectors

Euclidean Distance

The **Euclidean distance** between vectors $\mathbf{u}$ and $\mathbf{v}$ is:

Formula:

$$d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{n} (u_i - v_i)^2}$$

Properties:

- $d(\mathbf{u}, \mathbf{v}) \geq 0$ (non-negative)
- $d(\mathbf{u}, \mathbf{v}) = 0$ iff $\mathbf{u} = \mathbf{v}$ (identity)
- $d(\mathbf{u}, \mathbf{v}) = d(\mathbf{v}, \mathbf{u})$ (symmetric)
- $d(\mathbf{u}, \mathbf{w}) \leq d(\mathbf{u}, \mathbf{v}) + d(\mathbf{v}, \mathbf{w})$ (triangle inequality)

Example 3.7: Distance Calculation

Given:

$$\mathbf{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 4 \\ 6 \\ 7 \end{bmatrix}$$

Step 1: Compute difference

$$\mathbf{u} - \mathbf{v} = \begin{bmatrix} 1 - 4 \\ 2 - 6 \\ 3 - 7 \end{bmatrix} = \begin{bmatrix} -3 \\ -4 \\ -4 \end{bmatrix}$$

Step 2: Square each component $(-3)^2 = 9$, $(-4)^2 = 16$, $(-4)^2 = 16$

Step 3: Sum $9 + 16 + 16 = 41$

Step 4: Square root $d(\mathbf{u}, \mathbf{v}) = \sqrt{41} \approx 6.403$

Example 3.8: k-Nearest Neighbors (k-NN) Classification

Scenario: Classify iris flowers based on measurements.
Features: [sepal length, sepal width, petal length, petal width]
Training Data:

$$\begin{aligned}
\mathbf{x}_1 &= [5.1, 3.5, 1.4, 0.2]^T \quad \text{(Class: Setosa)} \\
\mathbf{x}_2 &= [6.7, 3.0, 5.2, 2.3]^T \quad \text{(Class: Virginica)} \\
\mathbf{x}_3 &= [5.0, 3.4, 1.5, 0.2]^T \quad \text{(Class: Setosa)} \\
\mathbf{x}_4 &= [5.9, 3.0, 5.1, 1.8]^T \quad \text{(Class: Virginica)}
\end{aligned}$$

Query (new flower): $\mathbf{q} = [5.2, 3.6, 1.4, 0.3]^T$ (Class: ?)
Step 1: Calculate distances to all training points
Distance to $\mathbf{x}_1$:

$$\begin{aligned}
d(\mathbf{q}, \mathbf{x}_1) &= \sqrt{(5.2-5.1)^2 + (3.6-3.5)^2 + (1.4-1.4)^2 + (0.3-0.2)^2} \\
&= \sqrt{0.01 + 0.01 + 0 + 0.01} = \sqrt{0.03} \approx 0.173
\end{aligned}$$

Distance to $\mathbf{x}_2$:

$$\begin{aligned}
d(\mathbf{q}, \mathbf{x}_2) &= \sqrt{(5.2-6.7)^2 + (3.6-3.0)^2 + (1.4-5.2)^2 + (0.3-2.3)^2} \\
&= \sqrt{2.25 + 0.36 + 14.44 + 4.00} = \sqrt{21.05} \approx 4.588
\end{aligned}$$

Distance to $\mathbf{x}_3$:

$$\begin{aligned}
d(\mathbf{q}, \mathbf{x}_3) &= \sqrt{(5.2-5.0)^2 + (3.6-3.4)^2 + (1.4-1.5)^2 + (0.3-0.2)^2} \\
&= \sqrt{0.04 + 0.04 + 0.01 + 0.01} = \sqrt{0.10} \approx 0.316
\end{aligned}$$

Distance to $\mathbf{x}_4$:

$$\begin{aligned}
d(\mathbf{q}, \mathbf{x}_4) &= \sqrt{(5.2-5.9)^2 + (3.6-3.0)^2 + (1.4-5.1)^2 + (0.3-1.8)^2} \\
&= \sqrt{0.49 + 0.36 + 13.69 + 2.25} = \sqrt{16.79} \approx 4.098
\end{aligned}$$

Step 2: Sort by distance

Neighbor	Distance	Class
$\mathbf{x}_1$	0.173	Setosa
$\mathbf{x}_3$	0.316	Setosa
$\mathbf{x}_4$	4.098	Virginica
$\mathbf{x}_2$	4.588	Virginica

Step 3: k=3 vote

- 3 nearest neighbors: $\mathbf{x}_1$ (Setosa), $\mathbf{x}_3$ (Setosa), $\mathbf{x}_4$ (Virginica)
- Setosa: 2 votes, Virginica: 1 vote

Prediction: Setosa

Example 3.9: K-Means Clustering

Scenario: Segment customers into groups for marketing.
Features: [annual spending, visit frequency]
Data points:

$$\mathbf{p}_1 = [100, 5]^T, \quad \mathbf{p}_2 = [120, 7]^T, \quad \mathbf{p}_3 = [800, 40]^T, \quad \mathbf{p}_4 = [850, 45]^T$$

Initial centroids (k=2):

$$\boldsymbol{\mu}_1 = [110, 6]^T, \quad \boldsymbol{\mu}_2 = [825, 42.5]^T$$

Step 1: Assign to nearest centroid
For $\mathbf{p}_1$:

$$d(\mathbf{p}_1, \boldsymbol{\mu}_1) = \sqrt{(100-110)^2 + (5-6)^2} = \sqrt{101} \approx 10.05$$
$$d(\mathbf{p}_1, \boldsymbol{\mu}_2) = \sqrt{(100-825)^2 + (5-42.5)^2} = \sqrt{526906.25} \approx 726.0$$

Assign to cluster 1 (closer to $\boldsymbol{\mu}_1$)
For $\mathbf{p}_3$:

$$d(\mathbf{p}_3, \boldsymbol{\mu}_1) = \sqrt{(800-110)^2 + (40-6)^2} = \sqrt{477256} \approx 690.9$$
$$d(\mathbf{p}_3, \boldsymbol{\mu}_2) = \sqrt{(800-825)^2 + (40-42.5)^2} = \sqrt{631.25} \approx 25.12$$

Assign to cluster 2 (closer to $\boldsymbol{\mu}_2$)
Assignments:

- Cluster 1: $\{\mathbf{p}_1, \mathbf{p}_2\}$ (low spenders)
- Cluster 2: $\{\mathbf{p}_3, \mathbf{p}_4\}$ (high spenders)

Step 2: Update centroids (mean of assigned points)

$$\boldsymbol{\mu}_1^{\text{new}} = \frac{\mathbf{p}_1 + \mathbf{p}_2}{2} = \frac{1}{2}\begin{bmatrix} 100+120 \\ 5+7 \end{bmatrix} = \begin{bmatrix} 110 \\ 6 \end{bmatrix}$$

$$\boldsymbol{\mu}_2^{\text{new}} = \frac{\mathbf{p}_3 + \mathbf{p}_4}{2} = \frac{1}{2}\begin{bmatrix} 800+850 \\ 40+45 \end{bmatrix} = \begin{bmatrix} 825 \\ 42.5 \end{bmatrix}$$

Converged! (centroids unchanged)
Business insight: Target low-spenders with promotions, give VIP treatment to high-spenders!

4 Vector Operations

4.1 Scalar-Vector Operations

4.1.1 Scalar Addition and Subtraction

Scalar-Vector Addition

Adding a scalar c to a vector $\mathbf{v}$ adds c to every component.

Formula:

$$\mathbf{v} + c = \begin{bmatrix} v_1 + c \\ v_2 + c \\ \vdots \\ v_n + c \end{bmatrix}$$

Geometric effect: Shifts the vector parallel to all axes equally.

Example 4.1: Scalar Addition

Given: $\mathbf{v} = \begin{bmatrix} 2 \\ 5 \\ -3 \end{bmatrix}$, $c = 10$

Calculate:

$$\mathbf{v} + c = \begin{bmatrix} 2 + 10 \\ 5 + 10 \\ -3 + 10 \end{bmatrix} = \begin{bmatrix} 12 \\ 15 \\ 7 \end{bmatrix}$$

Example 4.2: Bias Addition in Neural Networks

Scenario: Adding bias term after weighted sum.

Pre-activation (before bias):

$$\mathbf{z}_{\text{pre}} = \mathbf{W}\mathbf{x} = \begin{bmatrix} 0.5 \\ -0.3 \\ 0.8 \end{bmatrix}$$

Bias: $b = 0.1$ (scalar)

Add bias:

$$\mathbf{z} = \mathbf{z}_{\text{pre}} + b = \begin{bmatrix} 0.5 \\ -0.3 \\ 0.8 \end{bmatrix} + 0.1 = \begin{bmatrix} 0.6 \\ -0.2 \\ 0.9 \end{bmatrix}$$

Effect: Bias shifts the activation threshold, allowing the neuron to activate even when input is zero.

Why crucial: Without bias, $f(0) = 0$ always—network can't learn offsets!

Example 4.3: Data Centering (Mean Subtraction)

Scenario: Preprocessing data for PCA or neural networks.
Original data (5 samples, 1 feature):

$$\mathbf{x} = \begin{bmatrix} 10 \\ 12 \\ 8 \\ 14 \\ 11 \end{bmatrix}$$

Step 1: Calculate mean $\mu = \frac{1}{5}(10 + 12 + 8 + 14 + 11) = \frac{55}{5} = 11$
Step 2: Center data (subtract mean)

$$\mathbf{x}_{\text{centered}} = \mathbf{x} - \mu = \begin{bmatrix} 10 - 11 \\ 12 - 11 \\ 8 - 11 \\ 14 - 11 \\ 11 - 11 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \\ -3 \\ 3 \\ 0 \end{bmatrix}$$

Verification: New mean is zero! $\frac{1}{5}(-1 + 1 + (-3) + 3 + 0) = 0$ ✓
Why important:

- PCA requires centered data
- Faster convergence in gradient descent
- Prevents numerical issues

4.1.2 Scalar Multiplication and Division

Scalar-Vector Multiplication

Multiplying a vector $\mathbf{v}$ by scalar k scales every component.
Formula:

$$k\mathbf{v} = \begin{bmatrix} kv_1 \\ kv_2 \\ \vdots \\ kv_n \end{bmatrix}$$

Properties:

- $\|k\mathbf{v}\| = |k|\|\mathbf{v}\|$ (magnitude scales by $|k|$)
- Direction preserved if $k > 0$, reversed if $k < 0$
- Distributive: $k(\mathbf{u} + \mathbf{v}) = k\mathbf{u} + k\mathbf{v}$

Example 4.4: Scalar Multiplication

Given: $\mathbf{v} = \begin{bmatrix} 2 \\ -3 \\ 1 \end{bmatrix}$, $k = -0.5$

Calculate:

$$k\mathbf{v} = -0.5 \times \begin{bmatrix} 2 \\ -3 \\ 1 \end{bmatrix} = \begin{bmatrix} -1.0 \\ 1.5 \\ -0.5 \end{bmatrix}$$

Check magnitude:

$$\|\mathbf{v}\| = \sqrt{4+9+1} = \sqrt{14} \approx 3.742$$
$$\|k\mathbf{v}\| = \sqrt{1+2.25+0.25} = \sqrt{3.5} \approx 1.871$$
$$|k|\|\mathbf{v}\| = 0.5 \times 3.742 = 1.871 \quad \checkmark$$

Example 4.5: Learning Rate in Gradient Descent

Scenario: Updating weights during training.
Current weights:

$$\mathbf{w} = \begin{bmatrix} 0.5 \\ 0.3 \\ -0.2 \end{bmatrix}$$

Computed gradient:

$$\nabla L = \begin{bmatrix} 0.1 \\ -0.05 \\ 0.08 \end{bmatrix}$$

Learning rate: $\alpha = 0.01$
Gradient Descent Update: $\mathbf{w}_{\text{new}} = \mathbf{w} - \alpha\nabla L$
Step 1: Scale gradient by learning rate

$$\alpha\nabla L = 0.01 \times \begin{bmatrix} 0.1 \\ -0.05 \\ 0.08 \end{bmatrix} = \begin{bmatrix} 0.001 \\ -0.0005 \\ 0.0008 \end{bmatrix}$$

Step 2: Update weights

$$\mathbf{w}_{\text{new}} = \begin{bmatrix} 0.5 \\ 0.3 \\ -0.2 \end{bmatrix} - \begin{bmatrix} 0.001 \\ -0.0005 \\ 0.0008 \end{bmatrix} = \begin{bmatrix} 0.499 \\ 0.3005 \\ -0.2008 \end{bmatrix}$$

Effect of learning rate:

- α too large (0.5): Overshoots, diverges
- α too small (0.00001): Too slow, never converges
- α just right (0.01): Smooth convergence

This is scalar multiplication at the heart of all deep learning!

4.2 Vector-Vector Operations

4.2.1 Element-wise Addition and Subtraction

Vector Addition

Adding two vectors combines corresponding components.

Formula:

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} u_1 + v_1 \\ u_2 + v_2 \\ \vdots \\ u_n + v_n \end{bmatrix}$$

Requirement: Vectors must have same dimension!

Properties:

- Commutative: $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
- Associative: $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$
- Identity: $\mathbf{v} + \mathbf{0} = \mathbf{v}$

Example 4.6: Vector Addition

Given:

$$\mathbf{u} = \begin{bmatrix} 2 \\ 5 \\ -1 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 3 \\ -2 \\ 4 \end{bmatrix}$$

Calculate:

$$\mathbf{u} + \mathbf{v} = \begin{bmatrix} 2+3 \\ 5+(-2) \\ -1+4 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \\ 3 \end{bmatrix}$$

Subtraction:

$$\mathbf{u} - \mathbf{v} = \begin{bmatrix} 2-3 \\ 5-(-2) \\ -1-4 \end{bmatrix} = \begin{bmatrix} -1 \\ 7 \\ -5 \end{bmatrix}$$

Example 4.7: Residual Connections (ResNet)

Scenario: Skip connection in deep neural network.
Input to residual block:

$$\mathbf{x} = \begin{bmatrix} 0.5 \\ 0.3 \\ -0.2 \\ 0.8 \end{bmatrix}$$

Block transformation: $F(\mathbf{x})$ (two Conv layers + ReLU)

$$F(\mathbf{x}) = \begin{bmatrix} 0.1 \\ 0.05 \\ -0.15 \\ 0.2 \end{bmatrix}$$

Residual connection (skip): $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$
Calculate:

$$\mathbf{y} = \begin{bmatrix} 0.1 \\ 0.05 \\ -0.15 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 0.5 \\ 0.3 \\ -0.2 \\ 0.8 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.35 \\ -0.35 \\ 1.0 \end{bmatrix}$$

Why this matters:

- **Gradient flow:** Gradients flow directly through skip connection
- **Enables deep networks:** Can train 100+ layer networks
- **Identity mapping:** If $F(\mathbf{x}) \approx \mathbf{0}$, learns identity

Backpropagation through residual:

$$\frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \left(1 + \frac{\partial F}{\partial \mathbf{x}}\right)$$

The "+1" ensures gradients always flow back!

4.3 Dot Product (Inner Product)

Dot Product

The **dot product** (inner product, scalar product) of vectors $\mathbf{u}$ and $\mathbf{v}$ produces a **scalar**.
Algebraic Formula:

$$\mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n = \sum_{i=1}^{n} u_i v_i$$

Geometric Formula:

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos\theta$$

where θ is the angle between vectors.
Matrix Form: $\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T \mathbf{v}$

Property 4.3.1 (Dot Product Properties)**.** 1. **Commutative:** $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$

2. **Distributive:** $\mathbf{u} \cdot (\mathbf{v} + \mathbf{w}) = \mathbf{u} \cdot \mathbf{v} + \mathbf{u} \cdot \mathbf{w}$

3. **Scalar multiplication:** $(k\mathbf{u}) \cdot \mathbf{v} = k(\mathbf{u} \cdot \mathbf{v})$

4. **Self dot product:** $\mathbf{v} \cdot \mathbf{v} = \|\mathbf{v}\|^2$

Example 4.8: Computing Dot Product

Given:

$$\mathbf{u} = \begin{bmatrix} 2 \\ 3 \\ -1 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 1 \\ 4 \\ 5 \end{bmatrix}$$

Method 1: Component-wise multiplication

$$\mathbf{u} \cdot \mathbf{v} = (2)(1) + (3)(4) + (-1)(5) = 2 + 12 + (-5) = 9$$

Method 2: Matrix multiplication (row $\times$ column)

$$\mathbf{u}^T\mathbf{v} = [2, 3, -1] \begin{bmatrix} 1 \\ 4 \\ 5 \end{bmatrix} = 9$$

Dot Product Interpretation

Sign indicates relationship:

- $\mathbf{u} \cdot \mathbf{v} > 0$: Acute angle ($\theta < 90$) — similar direction
- $\mathbf{u} \cdot \mathbf{v} = 0$: Orthogonal ($\theta = 90$) — perpendicular
- $\mathbf{u} \cdot \mathbf{v} < 0$: Obtuse angle ($\theta > 90$) — opposite direction

Magnitude indicates alignment strength:

- Large positive: Strong alignment
- Near zero: Nearly perpendicular
- Large negative: Strong opposition

Example 4.9: Linear Regression Prediction

Scenario: Predicting house price.
Features:

$$\mathbf{x} = \begin{bmatrix} 1500 \\ 3 \\ 2 \\ 15 \end{bmatrix} = \begin{bmatrix} \text{sq ft} \\ \text{bedrooms} \\ \text{bathrooms} \\ \text{age (years)} \end{bmatrix}$$

Learned weights:

$$\mathbf{w} = \begin{bmatrix} 100 \\ 20000 \\ 15000 \\ -500 \end{bmatrix}$$

Bias: $b = 50000$
Prediction formula: $\hat{y} = \mathbf{w}^T\mathbf{x} + b = \mathbf{w} \cdot \mathbf{x} + b$
Step-by-step calculation:

$$\begin{aligned} \mathbf{w} \cdot \mathbf{x} &= (100)(1500) + (20000)(3) + (15000)(2) + (-500)(15) \\ &= 150000 + 60000 + 30000 + (-7500) = 232500 \end{aligned}$$

Add bias: $\hat{y} = 232500 + 50000 = \$282,500$
Interpretation of weights:

- Each sq ft adds \$100
- Each bedroom adds \$20,000
- Each bathroom adds \$15,000
- Each year of age subtracts \$500 (depreciation)

This single dot product is the entire prediction!

Example 4.10: Neural Network Neuron

Scenario: Single neuron deciding "Approve loan?"

Input features:

$$\mathbf{x} = \begin{bmatrix} 0.65 \\ 0.82 \\ 0.45 \end{bmatrix} = \begin{bmatrix} \text{credit score (normalized)} \\ \text{income (normalized)} \\ \text{debt ratio} \end{bmatrix}$$

Neuron weights:

$$\mathbf{w} = \begin{bmatrix} 0.8 \\ 0.6 \\ -0.9 \end{bmatrix}$$

Bias: $b = -0.2$

Step 1: Weighted sum (dot product)

$$\begin{aligned} z = \mathbf{w} \cdot \mathbf{x} + b &= (0.8)(0.65) + (0.6)(0.82) + (-0.9)(0.45) + (-0.2) \\ &= 0.52 + 0.492 + (-0.405) + (-0.2) = 0.407 \end{aligned}$$

Step 2: Activation function (sigmoid)

$$a = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-0.407}} = \frac{1}{1 + 0.666} = 0.600$$

Decision: $a = 0.600 > 0.5 \rightarrow$ **Approve loan** (60% confidence)

Weight interpretation:

- High credit score (+0.8): Strongly favors approval
- High income (+0.6): Moderately favors approval
- High debt ratio (-0.9): Strongly opposes approval

Every neuron in every neural network does this!

Example 4.11: Recommendation System (Cosine Similarity)

Scenario: Netflix recommending movies.
User A's ratings: [Action, Comedy, Drama, Horror, Sci-Fi]

$$\mathbf{u}_A = \begin{bmatrix} 5 \\ 2 \\ 4 \\ 1 \\ 5 \end{bmatrix}$$

User B's ratings:

$$\mathbf{u}_B = \begin{bmatrix} 4 \\ 3 \\ 3 \\ 2 \\ 4 \end{bmatrix}$$

Calculate similarity:
Step 1: Dot product

$$\begin{aligned} \mathbf{u}_A \cdot \mathbf{u}_B &= (5)(4) + (2)(3) + (4)(3) + (1)(2) + (5)(4) \\ &= 20 + 6 + 12 + 2 + 20 = 60 \end{aligned}$$

Step 2: Magnitudes

$$\begin{aligned} \|\mathbf{u}_A\| &= \sqrt{5^2 + 2^2 + 4^2 + 1^2 + 5^2} = \sqrt{71} \approx 8.426 \\ \|\mathbf{u}_B\| &= \sqrt{4^2 + 3^2 + 3^2 + 2^2 + 4^2} = \sqrt{54} \approx 7.348 \end{aligned}$$

Step 3: Cosine similarity

$$\text{similarity} = \frac{\mathbf{u}_A \cdot \mathbf{u}_B}{\|\mathbf{u}_A\|\|\mathbf{u}_B\|} = \frac{60}{8.426 \times 7.348} = \frac{60}{61.92} \approx 0.969$$

Interpretation:

- Similarity = 0.969 (scale: -1 to 1)
- Very high! Users have similar taste
- Recommend User B's favorite movies to User A

Why cosine similarity?

- Ignores magnitude (User A rates 1-5, User B rates 1-10 — still comparable)
- Focuses on preference pattern/direction
- Range [-1, 1] easier to interpret than raw dot product

5 Summary and Key Formulas

Essential Formulas Reference

Norms:

$$\|\mathbf{v}\|_1 = \sum |v_i| \quad \text{(L1 norm)}$$
$$\|\mathbf{v}\|_2 = \sqrt{\sum v_i^2} \quad \text{(L2 norm)}$$
$$\|\mathbf{v}\|_\infty = \max |v_i| \quad \text{(L}\infty\text{ norm)}$$

Distance: $d(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum (u_i - v_i)^2}$
Unit vector: $\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$
Dot product: $\mathbf{u} \cdot \mathbf{v} = \sum u_i v_i = \|\mathbf{u}\|\|\mathbf{v}\| \cos\theta$
Cosine similarity: $\cos\theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$

These 4 sections form the foundation of all ML/DL mathematics!

Master these concepts and the rest becomes much easier.

6 Linear Independence and Basis

This section explores how vectors relate to each other and how they can form coordinate systems. Understanding linear independence and basis vectors is crucial for feature engineering, dimensionality reduction, and understanding the capacity of neural networks.

6.1 Linear Combinations

Linear Combination

A **linear combination** of vectors $\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k$ is any vector of the form:

$$\mathbf{w} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k$$

where $c_1, c_2, \ldots, c_k$ are scalars (called **coefficients** or **weights**).
In matrix notation:

$$\mathbf{w} = \begin{bmatrix} \mathbf{v}_1 & \mathbf{v}_2 & \cdots & \mathbf{v}_k \end{bmatrix} \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_k \end{bmatrix}$$

Key Concepts:

- Linear combinations create new vectors from existing ones
- The set of all linear combinations forms a subspace
- Neural networks compute linear combinations followed by nonlinear activations
- Feature engineering often involves linear combinations of original features

Example 6.1.1: Creating Vectors from Linear Combinations

Problem: Given $\mathbf{v}_1 = [1, 0]$ and $\mathbf{v}_2 = [0, 1]$, express $\mathbf{w} = [3, 5]$ as a linear combination.
Solution:
We need to find c_1, c_2 such that:

$$\mathbf{w} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2$$

Step 1: Set up equation

$$\begin{bmatrix}3\\5\end{bmatrix} = c_1\begin{bmatrix}1\\0\end{bmatrix} + c_2\begin{bmatrix}0\\1\end{bmatrix}$$

Step 2: Component-wise equations

$$c_1(1) + c_2(0) = 3 \Rightarrow c_1 = 3$$

$$c_1(0) + c_2(1) = 5 \Rightarrow c_2 = 5$$

Step 3: Solution

$$\mathbf{w} = 3\mathbf{v}_1 + 5\mathbf{v}_2 = 3[1, 0] + 5[0, 1] = [3, 5] \quad \checkmark$$

Interpretation: Any 2D vector can be expressed as a linear combination of standard basis vectors.

Example 6.1.2: Linear Combination in 3D

Problem: Express $\mathbf{b} = [7, 3, 1]$ as a linear combination of:

$$\mathbf{u}_1 = [1, 1, 0], \quad \mathbf{u}_2 = [1, 0, 1], \quad \mathbf{u}_3 = [0, 1, 1]$$

Solution:
Step 1: Set up system

$$c_1\begin{bmatrix}1\\1\\0\end{bmatrix} + c_2\begin{bmatrix}1\\0\\1\end{bmatrix} + c_3\begin{bmatrix}0\\1\\1\end{bmatrix} = \begin{bmatrix}7\\3\\1\end{bmatrix}$$

Step 2: Component equations

$$\begin{aligned} c_1 + c_2 &= 7 \quad \text{(first component)} \\ c_1 + c_3 &= 3 \quad \text{(second component)} \\ c_2 + c_3 &= 1 \quad \text{(third component)} \end{aligned}$$

Step 3: Solve system From equation 3: $c_2 = 1 - c_3$
Substitute into equation 1: $c_1 + (1 - c_3) = 7 \Rightarrow c_1 = 6 + c_3$
Substitute into equation 2: $(6 + c_3) + c_3 = 3 \Rightarrow 2c_3 = -3 \Rightarrow c_3 = -1.5$
Back-substitute:

$$c_1 = 6 + (-1.5) = 4.5, \quad c_2 = 1 - (-1.5) = 2.5$$

Step 4: Verification

$$4.5[1, 1, 0] + 2.5[1, 0, 1] + (-1.5)[0, 1, 1] = [4.5 + 2.5, 4.5 - 1.5, 2.5 - 1.5] = [7, 3, 1] \quad \checkmark$$

Solution: $\mathbf{b} = 4.5\mathbf{u}_1 + 2.5\mathbf{u}_2 - 1.5\mathbf{u}_3$

ML Application 6.1.1: Feature Engineering

Scenario: Real estate price prediction with engineered features.
Original features:

- f_1: Square footage
- f_2: Number of bedrooms
- f_3: Lot size

Feature Engineering: Create new feature using linear combination

$$f_{\text{new}} = 0.6 \cdot f_1 + 0.3 \cdot f_2 + 0.1 \cdot f_3$$

Example Calculation:
House A: Square footage = 2000, Bedrooms = 3, Lot size = 5000
Step 1: Apply linear combination

$$f_{\text{new}} = 0.6(2000) + 0.3(3) + 0.1(5000)$$

$$= 1200 + 0.9 + 500 = 1700.9$$

Interpretation: The engineered feature combines size metrics with learned weights, potentially capturing a "overall property value indicator" that improves model performance.
Why This Works:

- Linear combinations can capture domain knowledge
- Reduces dimensionality while preserving information
- Similar to what neural networks learn automatically

6.2 Linear Independence

Linear Independence

A set of vectors $\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k\}$ is **linearly independent** if the only solution to:

$$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k = \mathbf{0}$$

is the trivial solution: $c_1 = c_2 = \cdots = c_k = 0$.
Equivalently, no vector in the set can be expressed as a linear combination of the others.
If a non-trivial solution exists, the vectors are **linearly dependent**.

Testing for Linear Independence:

For n vectors in $\mathbb{R}^n$:

1. Form matrix $A = [\mathbf{v}_1\ \mathbf{v}_2\ \cdots\ \mathbf{v}_n]$
2. Calculate determinant: $\det(A) \neq 0$ implies independence
3. Or: Row reduce to check for pivots in each column

Important Properties:

- At most n vectors in $\mathbb{R}^n$ can be independent
- If one vector is $\mathbf{0}$, the set is dependent
- If any two vectors are parallel, the set is dependent
- Independent vectors span a space efficiently (no redundancy)

Example 6.2.1: Testing Independence in 2D

Problem: Are $\mathbf{v}_1 = [1, 2]$ and $\mathbf{v}_2 = [2, 4]$ linearly independent?
Solution:
Method 1: Direct Test
Check if $c_1\mathbf{v}_1 + c_2\mathbf{v}_2 = \mathbf{0}$ has only trivial solution:

$$c_1[1, 2] + c_2[2, 4] = [0, 0]$$

$$[c_1 + 2c_2, 2c_1 + 4c_2] = [0, 0]$$

From first component: $c_1 = -2c_2$
Substitute into second: $2(-2c_2) + 4c_2 = -4c_2 + 4c_2 = 0$ (satisfied for any c_2)
Choose $c_2 = 1$, then $c_1 = -2$ (non-trivial solution exists!)
Conclusion: Linearly dependent because $-2\mathbf{v}_1 + 1\mathbf{v}_2 = \mathbf{0}$
Method 2: Determinant

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}, \quad \det(A) = (1)(4) - (2)(2) = 0$$

$\det(A) = 0$ confirms dependence.
Geometric Interpretation: $\mathbf{v}_2 = 2\mathbf{v}_1$ (parallel vectors).

Example 6.2.2: Independence in 3D

Problem: Are these vectors linearly independent?

$$\mathbf{u}_1 = [1, 0, 1], \quad \mathbf{u}_2 = [0, 1, 0], \quad \mathbf{u}_3 = [1, 1, 0]$$

Solution:
Step 1: Form matrix and calculate determinant

$$A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Step 2: Calculate determinant (cofactor expansion along row 1)

$$\det(A) = 1 \cdot \begin{vmatrix} 1 & 1 \\ 0 & 0 \end{vmatrix} - 0 + 1 \cdot \begin{vmatrix} 0 & 1 \\ 1 & 0 \end{vmatrix}$$

$$= 1(0 - 0) + 1(0 - 1) = -1 \neq 0$$

Conclusion: Linearly independent because $\det(A) \neq 0$.
Verification: Try to express $\mathbf{u}_3$ as combination of $\mathbf{u}_1, \mathbf{u}_2$:

$$\mathbf{u}_3 = c_1\mathbf{u}_1 + c_2\mathbf{u}_2 \Rightarrow [1, 1, 0] = c_1[1, 0, 1] + c_2[0, 1, 0]$$

This gives: $c_1 = 1, c_2 = 1, 0 = c_1$ (contradiction!) Confirms independence.

ML Application 6.2.1: Multicollinearity in Linear Regression

Scenario: Predicting student grades with features:

- x_1: Study hours per week
- x_2: Quiz scores (average)
- x_3: 2× study hours per week (accidentally included!)

Problem: Features are linearly dependent: $\mathbf{x}_3 = 2\mathbf{x}_1$

Impact on Regression:

The design matrix becomes:

$$X = \begin{bmatrix} 10 & 85 & 20 \\ 15 & 90 & 30 \\ 8 & 78 & 16 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Mathematical Issue:

Normal equation: $\mathbf{w} = (X^T X)^{-1} X^T \mathbf{y}$

But $X^T X$ is singular (non-invertible) because columns are dependent!

Consequences:

1. No unique solution for weights
2. Regression coefficients become unstable
3. Small data changes cause huge weight swings
4. Statistical inference (p-values, confidence intervals) breaks down

Detection: Calculate Variance Inflation Factor (VIF):

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

where R_j^2 is R^2 from regressing x_j on other features.

VIF > 10 indicates multicollinearity.

Solutions:

- Remove redundant features (x_3)
- Use Ridge Regression (L2 regularization)
- Apply PCA for dimensionality reduction
- Use feature selection algorithms

6.3 Span

Span

The **span** of a set of vectors $\{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k\}$ is the set of all possible linear combinations:

$$\text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_k\} = \{c_1\mathbf{v}_1 + \cdots + c_k\mathbf{v}_k \mid c_i \in \mathbb{R}\}$$

The span forms a **subspace** of the ambient vector space.

Key Properties:

- Span is always a subspace (closed under addition and scalar multiplication)
- dim(span) = number of linearly independent vectors

- If $\text{span}\{\mathbf{v}_1, \ldots, \mathbf{v}_n\} = \mathbb{R}^n$, the vectors **span the space**
- Adding dependent vectors doesn't change the span

Geometric Interpretation:

- 1 vector: spans a line through origin
- 2 independent vectors in $\mathbb{R}^3$: span a plane
- 3 independent vectors in $\mathbb{R}^3$: span all of $\mathbb{R}^3$

Example 6.3.1: Span of Two Vectors in 2D

Problem: What is $\text{span}\{[1,0],[0,1]\}$?
Solution:
Any vector in the span has the form:

$$\mathbf{v} = c_1[1,0] + c_2[0,1] = [c_1, c_2]$$

Since c_1, c_2 can be any real numbers:
Answer: $\text{span}\{[1,0],[0,1]\} = \mathbb{R}^2$ (entire 2D plane)
Verification: Take any point $[a,b]$:

$$[a,b] = a[1,0] + b[0,1] \quad \checkmark$$

Example 6.3.2: Span of Dependent Vectors

Problem: What is $\text{span}\{[1,2],[2,4]\}$?
Solution:
Step 1: Check dependence Since $[2,4] = 2[1,2]$, vectors are dependent.
Step 2: Determine span

$$c_1[1,2] + c_2[2,4] = c_1[1,2] + c_2 \cdot 2[1,2] = (c_1 + 2c_2)[1,2]$$

Let $k = c_1 + 2c_2$, then span $= \{k[1,2] \mid k \in \mathbb{R}\}$
Answer: A line through origin with direction $[1,2]$ (1-dimensional subspace)
Parametric form: $\{[t,2t] \mid t \in \mathbb{R}\}$ or $y = 2x$

ML Application 6.3.1: Feature Space and Model Capacity

Scenario: Linear classifier for 2D data.
Dataset: Points in $\mathbb{R}^2$ with binary labels.
Case 1: Features $\mathbf{x} = [x_1, x_2]$ (2 independent features)
Span: $\mathbb{R}^2$ (entire plane)
Decision boundary: $w_1x_1 + w_2x_2 + b = 0$ (any line)
Model Capacity: Can represent any linear decision boundary.
Case 2: Features $\mathbf{x} = [x_1, 2x_1]$ (dependent)
Span: Line $y = 2x$ (1D subspace)
Decision boundary: $w_1x_1 + w_2(2x_1) + b = (w_1 + 2w_2)x_1 + b = 0$
This only represents vertical lines! (parallel to y-axis)
Model Capacity: Severely limited, cannot learn general decision boundaries.
Lesson: Feature space dimensionality (= span dimension) determines model expressiveness. Always use linearly independent features for maximum model capacity.

6.4 Basis

Basis

A **basis** for a vector space V is a set of vectors $\{\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n\}$ that:

1. Are **linearly independent**
2. **Span** the entire space: $\text{span}\{\mathbf{b}_1, \ldots, \mathbf{b}_n\} = V$

Every vector $\mathbf{v} \in V$ can be uniquely expressed as:

$$\mathbf{v} = c_1\mathbf{b}_1 + c_2\mathbf{b}_2 + \cdots + c_n\mathbf{b}_n$$

where $(c_1, c_2, \ldots, c_n)$ are called the **coordinates** of $\mathbf{v}$ in this basis.
The number of vectors in any basis equals the **dimension** of V.

Standard Basis in $\mathbb{R}^n$:

$$\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix}, \quad \ldots, \quad \mathbf{e}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

Properties:

- Basis is minimal spanning set (remove any vector, can't span anymore)
- Basis is maximal independent set (add any vector, becomes dependent)
- All bases of same space have same number of vectors
- Coordinates in different bases represent same vector differently

Example 6.4.1: Verifying a Basis in $\mathbb{R}^2$

Problem: Show that $\mathcal{B} = \{[1, 1], [1, -1]\}$ is a basis for $\mathbb{R}^2$.
Solution:
Part 1: Linear Independence
Test: $c_1[1, 1] + c_2[1, -1] = [0, 0]$

$$[c_1 + c_2, c_1 - c_2] = [0, 0]$$

System:

$$c_1 + c_2 = 0$$
$$c_1 - c_2 = 0$$

Add equations: $2c_1 = 0 \Rightarrow c_1 = 0$
Substitute: $0 + c_2 = 0 \Rightarrow c_2 = 0$
Only trivial solution exists **Independent**
Part 2: Span
Show any $[x, y] \in \mathbb{R}^2$ can be expressed:

$$[x, y] = c_1[1, 1] + c_2[1, -1] = [c_1 + c_2, c_1 - c_2]$$

System:

$$c_1 + c_2 = x$$
$$c_1 - c_2 = y$$

Solution: $c_1 = \frac{x+y}{2}$, $c_2 = \frac{x-y}{2}$
Always has solution **Spans** $\mathbb{R}^2$
Conclusion: $\mathcal{B}$ is a basis for $\mathbb{R}^2$.
Example: Express $[5, 3]$ in basis $\mathcal{B}$:

$$c_1 = \frac{5+3}{2} = 4, \quad c_2 = \frac{5-3}{2} = 1$$

$$[5, 3] = 4[1, 1] + 1[1, -1] \quad \checkmark$$

ML Application 6.4.1: PCA - Finding New Basis

Scenario: 2D dataset with 100 points, highly correlated features.
Original basis: Standard $\{[1,0],[0,1]\}$
Data points (sample):

$$\mathbf{x}_1 = [2, 2.1],\ \mathbf{x}_2 = [3, 3.2],\ \mathbf{x}_3 = [4, 3.9],\ \ldots$$

Observation: Data lies approximately on line $y \approx x$ (high correlation).
PCA Process:
Step 1: Center data (subtract mean)
Mean: $\bar{\mathbf{x}} = [3, 3.067]$
Centered: $\tilde{\mathbf{x}}_1 = [-1, -0.967], \ldots$
Step 2: Compute covariance matrix

$$C = \frac{1}{n}\sum \tilde{\mathbf{x}}_i \tilde{\mathbf{x}}_i^T \approx \begin{bmatrix} 2.0 & 1.98 \\ 1.98 & 2.05 \end{bmatrix}$$

Step 3: Find eigenvectors (new basis directions)

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}}[1,1] \quad (\lambda_1 = 4.03,\ \text{variance along diagonal})$$

$$\mathbf{v}_2 = \frac{1}{\sqrt{2}}[1,-1] \quad (\lambda_2 = 0.02,\ \text{minimal variance})$$

Step 4: New basis $\mathcal{B}_{\text{PCA}} = \{\mathbf{v}_1, \mathbf{v}_2\}$
Result: In new basis:

- First component captures 99.5% of variance
- Second component: only 0.5%
- Can discard $\mathbf{v}_2$ with minimal information loss (dimensionality reduction!)

Interpretation: PCA finds an optimal basis where data is least correlated, enabling efficient compression.

7 Vector Spaces

Vector spaces provide the mathematical framework for understanding where vectors "live" and how they interact. This abstraction is crucial for machine learning theory, as it formalizes concepts like feature spaces, model representations, and transformations.

7.1 Subspaces

Subspace

A **subspace** W of a vector space V is a subset that is itself a vector space, satisfying:

1. **Contains zero vector:** $\mathbf{0} \in W$
2. **Closed under addition:** If $\mathbf{u}, \mathbf{v} \in W$, then $\mathbf{u} + \mathbf{v} \in W$
3. **Closed under scalar multiplication:** If $\mathbf{v} \in W$ and $c \in \mathbb{R}$, then $c\mathbf{v} \in W$

Equivalently: W is closed under linear combinations.

Common Examples:

In $\mathbb{R}^3$:

- Trivial subspace: $\{\mathbf{0}\}$
- Lines through origin: $\text{span}\{\mathbf{v}\}$ for any $\mathbf{v} \neq \mathbf{0}$
- Planes through origin: $\text{span}\{\mathbf{v}_1, \mathbf{v}_2\}$ (independent)
- Entire space: $\mathbb{R}^3$

Non-examples: Lines/planes not through origin are NOT subspaces!

Key Properties:

- Intersection of subspaces is a subspace
- Span of any set of vectors is a subspace
- Dimension of subspace $\leq$ dimension of full space

Example 7.1.1: Verifying a Subspace

Problem: Is $W = \{[x, 2x, 3x] \mid x \in \mathbb{R}\} \subseteq \mathbb{R}^3$ a subspace?
Solution:
Test 1: Zero vector When $x = 0$: $[0, 0, 0] \in W$
Test 2: Closure under addition Take $\mathbf{u} = [a, 2a, 3a]$ and $\mathbf{v} = [b, 2b, 3b]$ in W:

$$\mathbf{u} + \mathbf{v} = [a + b, 2a + 2b, 3a + 3b] = [a + b, 2(a + b), 3(a + b)]$$

This has form $[c, 2c, 3c]$ where $c = a + b$ Still in W
Test 3: Closure under scalar multiplication Take $\mathbf{u} = [a, 2a, 3a]$ in W and scalar k:

$$k\mathbf{u} = [ka, 2ka, 3ka] = [ka, 2(ka), 3(ka)]$$

This has form $[c, 2c, 3c]$ where $c = ka$ Still in W
Conclusion: W is a subspace of $\mathbb{R}^3$.
Geometric interpretation: W is a line through origin in direction $[1, 2, 3]$.

Example 7.1.2: Non-Subspace Example

Problem: Is $S = \{[x, y] \mid x + y = 1\} \subseteq \mathbb{R}^2$ a subspace?
Solution:
Test 1: Zero vector For $[0, 0]$ to be in S: $0 + 0 = 0 \neq 1$
Conclusion: S is NOT a subspace (fails zero vector test).
Geometric interpretation: S is the line $x + y = 1$, which doesn't pass through origin.
Additional verification - closure fails: Take $[1, 0] \in S$ and $[0, 1] \in S$:

$$[1, 0] + [0, 1] = [1, 1], \quad \text{but } 1 + 1 = 2 \neq 1$$

So $[1, 1] \notin S$ Not closed under addition.

ML Application 7.1.1: Null Space and Regularization

Scenario: Ridge regression with regularization.
Model: $\mathbf{y} = X\mathbf{w}$ where X is $m \times n$ design matrix.
Null Space: $\mathcal{N}(X) = \{\mathbf{w} \mid X\mathbf{w} = \mathbf{0}\}$
This is a subspace of $\mathbb{R}^n$ (weight space).
Example: $X = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$ (rank 1, dependent columns)
Find null space:

$$\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives: $w_1 + 2w_2 = 0 \Rightarrow w_1 = -2w_2$
Null space: $\mathcal{N}(X) = \text{span}\{[-2, 1]\}$ (1-dimensional subspace)
Interpretation: Any weight vector of form $[-2t, t]$ produces zero output!
This means:

- Multiple weight solutions exist (non-unique)
- Model is underdetermined
- Need regularization to select unique solution

Ridge solution: $(X^TX + \lambda I)^{-1}X^T\mathbf{y}$ makes matrix invertible, selecting smallest-norm solution.

7.2 Hyperplanes and Decision Boundaries

Hyperplane

A **hyperplane** in $\mathbb{R}^n$ is an $(n-1)$-dimensional flat subspace defined by:

$$\mathbf{w}^T\mathbf{x} + b = 0$$

or equivalently:

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = 0$$

where:

- $\mathbf{w} = [w_1, w_2, \ldots, w_n]^T$ is the **normal vector** (perpendicular to hyperplane)
- b is the **bias** (shifts hyperplane from origin)
- $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$ is any point on the hyperplane

Special cases:

- $\mathbb{R}^2$: Hyperplane is a line
- $\mathbb{R}^3$: Hyperplane is a plane
- $\mathbb{R}^n$: Hyperplane has dimension $n-1$

Distance from Point to Hyperplane:

For point $\mathbf{x}_0$ and hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$:

$$d = \frac{|\mathbf{w}^T\mathbf{x}_0 + b|}{\|\mathbf{w}\|}$$

Signed Distance (indicates which side):

$$d_{\text{signed}} = \frac{\mathbf{w}^T\mathbf{x}_0 + b}{\|\mathbf{w}\|}$$

- $d_{\text{signed}} > 0$: Point on positive side (direction of $\mathbf{w}$)
- $d_{\text{signed}} < 0$: Point on negative side
- $d_{\text{signed}} = 0$: Point on hyperplane

Example 7.2.1: Hyperplane in 2D

Problem: Consider hyperplane $2x + 3y - 6 = 0$.

1. Find normal vector and bias
2. Calculate distance from point $(1, 1)$ to hyperplane
3. Determine which side point $(0, 0)$ is on

Solution:

Part 1: Identify components

$$\mathbf{w} = [2, 3]^T, \quad b = -6$$

Normal vector points in direction $[2, 3]$.

Part 2: Distance from $(1, 1)$

$$d = \frac{|2(1) + 3(1) - 6|}{\sqrt{2^2 + 3^2}} = \frac{|2 + 3 - 6|}{\sqrt{13}} = \frac{|-1|}{\sqrt{13}} = \frac{1}{\sqrt{13}} \approx 0.277$$

Part 3: Side of origin $(0, 0)$

$$\text{signed distance} = \frac{2(0) + 3(0) - 6}{\sqrt{13}} = \frac{-6}{\sqrt{13}} < 0$$

Origin is on negative side (opposite direction of normal vector $[2, 3]$).

Verification: Find actual point on line: When $x = 0$: $3y = 6 \Rightarrow y = 2$, so $(0, 2)$ is on hyperplane.

Distance from $(1, 1)$ to $(0, 2)$: $\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414$

This is NOT perpendicular distance! Perpendicular gives $\frac{1}{\sqrt{13}} \approx 0.277$

Example 7.2.2: Hyperplane in 3D

Problem: Find equation of plane through point $(1, 2, 3)$ with normal vector $\mathbf{n} = [1, -1, 2]$.
Solution:
Step 1: General form

$$\mathbf{n}^T(\mathbf{x} - \mathbf{x}_0) = 0$$

$$[1, -1, 2] \cdot ([x, y, z] - [1, 2, 3]) = 0$$

Step 2: Expand

$$1(x - 1) + (-1)(y - 2) + 2(z - 3) = 0$$

$$x - 1 - y + 2 + 2z - 6 = 0$$

$$x - y + 2z - 5 = 0$$

Step 3: Standard form

$$\mathbf{w}^T\mathbf{x} + b = 0 \quad \text{where } \mathbf{w} = [1, -1, 2]^T,\ b = -5$$

Verification: Substitute $(1, 2, 3)$:

$$1(1) - 1(2) + 2(3) - 5 = 1 - 2 + 6 - 5 = 0 \quad \checkmark$$

ML Application 7.2.1: Linear Classifier (Binary Classification)

Scenario: Email spam detection with 2 features.
Features:

- x_1: Number of suspicious keywords
- x_2: Number of links

Learned decision boundary: $2x_1 + 3x_2 - 10 = 0$
This gives:

- Weight vector: $\mathbf{w} = [2, 3]$
- Bias: $b = -10$
- Classification rule: $f(\mathbf{x}) = \text{sign}(\mathbf{w}^T\mathbf{x} + b)$

Example Classifications:
Email 1: 3 keywords, 2 links $\rightarrow \mathbf{x}_1 = [3, 2]$

$$\mathbf{w}^T\mathbf{x}_1 + b = 2(3) + 3(2) - 10 = 6 + 6 - 10 = 2 > 0$$

Classification: Spam (positive class)
Distance from boundary:

$$d = \frac{2}{\sqrt{2^2 + 3^2}} = \frac{2}{\sqrt{13}} \approx 0.555$$

Email 2: 1 keyword, 1 link $\rightarrow \mathbf{x}_2 = [1, 1]$

$$\mathbf{w}^T\mathbf{x}_2 + b = 2(1) + 3(1) - 10 = -5 < 0$$

Classification: Not spam (negative class)
Distance from boundary:

$$d = \frac{|-5|}{\sqrt{13}} \approx 1.387$$

Confidence interpretation: Email 2 is farther from boundary (1.387 vs 0.555), so classifier is more confident it's not spam.
Decision boundary visualization:

$$x_2 = \frac{10 - 2x_1}{3}$$

Points above this line $\rightarrow$ spam; below $\rightarrow$ not spam.

ML Application 7.2.2: Support Vector Machines (SVM)

Scenario: SVM finds optimal hyperplane with maximum margin.
Training data (2D):

- Class +1: $\{[2,3],[3,3],[2,2]\}$
- Class -1: $\{[1,1],[0,1],[1,0]\}$

SVM Objective:

$$\text{Maximize margin } = \frac{2}{\|\mathbf{w}\|} \quad \text{subject to} \quad y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$$

Optimal solution found:

$$\mathbf{w} = [1,1]^T, \quad b = -2.5$$

Decision boundary: $x_1 + x_2 - 2.5 = 0$ or $x_2 = 2.5 - x_1$
Margin calculation:

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{1^2+1^2}} = \frac{2}{\sqrt{2}} = \sqrt{2} \approx 1.414$$

Support vectors (points on margin boundaries):

- From class +1: $[2,2]$ (closest to boundary)
- From class -1: $[1,1]$ (closest to boundary)

Verify margin constraints:
For $[2,2]$ (class +1):

$$1(2) + 1(2) - 2.5 = 1.5 \Rightarrow \frac{1.5}{\sqrt{2}} \approx 1.06 > 1 \quad \checkmark$$

For $[1,1]$ (class -1):

$$1(1) + 1(1) - 2.5 = -0.5 \Rightarrow \frac{|-0.5|}{\sqrt{2}} \approx 0.35$$

Geometric insight: SVM creates two parallel hyperplanes:

- $\mathbf{w}^T\mathbf{x} + b = +1$ (upper margin boundary)
- $\mathbf{w}^T\mathbf{x} + b = -1$ (lower margin boundary)
- Decision boundary: $\mathbf{w}^T\mathbf{x} + b = 0$ (midway between)

The perpendicular distance between margin boundaries is the margin width.

8 Advanced ML/DL Applications

This section explores sophisticated applications of vector concepts in modern machine learning and deep learning, including embedding spaces, attention mechanisms, and optimization algorithms.

8.1 Embedding Spaces

Embedding

An **embedding** is a mapping from a discrete space (e.g., words, images, users) to a continuous vector space $\mathbb{R}^d$:

$$\phi : \mathcal{X} \to \mathbb{R}^d$$

where d is the embedding dimension (typically 50-1000).

Properties of good embeddings:

- **Semantic similarity:** Similar items have similar vectors (measured by cosine similarity or distance)
- **Algebraic structure:** Relationships preserved through vector arithmetic
- **Dense representation:** Low-dimensional compared to original space

Common Embedding Types:

1. **Word Embeddings:** Word2Vec, GloVe, FastText
2. **Sentence/Document Embeddings:** Doc2Vec, BERT, sentence transformers
3. **Image Embeddings:** CNN features (ResNet, VGG)
4. **User/Item Embeddings:** Matrix factorization, collaborative filtering
5. **Graph Embeddings:** Node2Vec, GraphSAGE

ML Application 9.1.1: Word2Vec - Skip-gram Model

Scenario: Learning word embeddings from text corpus.
Training corpus: "the cat sat on the mat"
Skip-gram objective: Predict context words from center word.
For center word "cat", predict: "the", "sat"
Model:

$$P(\text{context} \mid \text{center}) = \frac{\exp(\mathbf{v}_{\text{context}}^T \mathbf{v}_{\text{center}})}{\sum_{w \in V} \exp(\mathbf{v}_w^T \mathbf{v}_{\text{center}})}$$

Learned embeddings (simplified 3D):

$$\begin{aligned}
\mathbf{v}_{\text{cat}} &= [0.5, 0.2, 0.8] \\
\mathbf{v}_{\text{dog}} &= [0.6, 0.1, 0.7] \\
\mathbf{v}_{\text{car}} &= [0.1, 0.8, 0.3] \\
\mathbf{v}_{\text{king}} &= [0.7, 0.3, 0.5] \\
\mathbf{v}_{\text{queen}} &= [0.6, 0.4, 0.4] \\
\mathbf{v}_{\text{man}} &= [0.8, 0.2, 0.6] \\
\mathbf{v}_{\text{woman}} &= [0.7, 0.3, 0.5]
\end{aligned}$$

Similarity calculations:
1. Cat vs Dog:

$$\cos(\mathbf{v}_{\text{cat}}, \mathbf{v}_{\text{dog}}) = \frac{0.5(0.6) + 0.2(0.1) + 0.8(0.7)}{\|\mathbf{v}_{\text{cat}}\| \|\mathbf{v}_{\text{dog}}\|}$$

$$= \frac{0.3 + 0.02 + 0.56}{\sqrt{0.93} \times \sqrt{0.86}} = \frac{0.88}{0.893} \approx 0.985$$

High similarity! (Both animals)
2. Cat vs Car:

$$\cos(\mathbf{v}_{\text{cat}}, \mathbf{v}_{\text{car}}) = \frac{0.5(0.1) + 0.2(0.8) + 0.8(0.3)}{\sqrt{0.93} \times \sqrt{0.74}}$$

$$= \frac{0.05 + 0.16 + 0.24}{0.829} \approx 0.543$$

Lower similarity (different categories)
3. Famous analogy: king - man + woman queen

$$\mathbf{v}_{\text{result}} = \mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}}$$

$$= [0.7, 0.3, 0.5] - [0.8, 0.2, 0.6] + [0.7, 0.3, 0.5]$$

$$= [0.6, 0.4, 0.4]$$

Find nearest vector:

$$\cos(\mathbf{v}_{\text{result}}, \mathbf{v}_{\text{queen}}) = \frac{[0.6, 0.4, 0.4] \cdot [0.6, 0.4, 0.4]}{\|\mathbf{v}_{\text{result}}\| \|\mathbf{v}_{\text{queen}}\|}$$

$$= \frac{0.36 + 0.16 + 0.16}{0.68 \times 0.68} = \frac{0.68}{0.68} = 1.0$$

Perfect match! Vector arithmetic captures gender relationships.
Geometric interpretation: The vector from "man" to "king" represents "royalty + male". Adding this to "woman" gives "royalty + female" = "queen".

ML Application 9.1.2: Recommendation System with User Embeddings

Scenario: Movie recommendation using collaborative filtering.

Setup:

- Users: 1000 people
- Movies: 5000 titles
- Embedding dimension: $d = 50$

Model: Matrix factorization

$$\text{Rating}_{ui} \approx \mathbf{p}_u^T \mathbf{q}_i$$

where:

- $\mathbf{p}_u \in \mathbb{R}^{50}$: User embedding
- $\mathbf{q}_i \in \mathbb{R}^{50}$: Movie embedding

Example (simplified to 3D):

User Alice: $\mathbf{p}_A = [0.8, 0.2, 0.5]$ (likes action, dislikes romance, neutral on comedy)

Movies:

$$\begin{aligned} \mathbf{q}_{\text{Action Movie}} &= [0.9, 0.1, 0.3] \\ \mathbf{q}_{\text{Rom-Com}} &= [0.2, 0.8, 0.7] \\ \mathbf{q}_{\text{Pure Action}} &= [1.0, 0.0, 0.2] \end{aligned}$$

Predicted ratings:

1. Action Movie:

$$\mathbf{p}_A^T \mathbf{q}_{\text{Action}} = 0.8(0.9) + 0.2(0.1) + 0.5(0.3) = 0.72 + 0.02 + 0.15 = 0.89$$

High score! Should recommend.

2. Rom-Com:

$$\mathbf{p}_A^T \mathbf{q}_{\text{Rom-Com}} = 0.8(0.2) + 0.2(0.8) + 0.5(0.7) = 0.16 + 0.16 + 0.35 = 0.67$$

Moderate score.

3. Pure Action:

$$\mathbf{p}_A^T \mathbf{q}_{\text{Pure Action}} = 0.8(1.0) + 0.2(0.0) + 0.5(0.2) = 0.8 + 0 + 0.1 = 0.9$$

Highest score! Top recommendation.

Interpretation: Dot product measures alignment between user preferences and movie characteristics. Higher values = better matches.

Training: Minimize squared error:

$$\min_{P,Q} \sum_{(u,i) \in \text{observed}} (r_{ui} - \mathbf{p}_u^T \mathbf{q}_i)^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

8.2 Attention Mechanisms

Attention Mechanism

Attention computes weighted combinations of vectors based on relevance scores.
Given:

- Query vector: $\mathbf{q} \in \mathbb{R}^{d_k}$
- Key vectors: $\mathbf{k}_1, \ldots, \mathbf{k}_n \in \mathbb{R}^{d_k}$
- Value vectors: $\mathbf{v}_1, \ldots, \mathbf{v}_n \in \mathbb{R}^{d_v}$

Scaled Dot-Product Attention:

$$\text{Attention}(\mathbf{q}, K, V) = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i$$

where attention weights are:

$$\alpha_i = \frac{\exp(\mathbf{q}^T \mathbf{k}_i / \sqrt{d_k})}{\sum_{j=1}^{n} \exp(\mathbf{q}^T \mathbf{k}_j / \sqrt{d_k})} = \text{softmax}\left(\frac{\mathbf{q}^T \mathbf{k}_i}{\sqrt{d_k}}\right)$$

Intuition:

- Dot product $\mathbf{q}^T \mathbf{k}_i$ measures relevance
- Softmax converts to probability distribution
- Scale by $\sqrt{d_k}$ prevents saturation
- Output is weighted sum of values

ML Application 9.2.1: Simple Attention Example

Scenario: Machine translation - translate "I love cats" to French.
Currently translating "love", need to attend to relevant input words.
Vectors (simplified 3D):
Query (from "love"): $\mathbf{q} = [0.8, 0.6, 0.3]$
Keys (from input):

$$\mathbf{k}_1(\text{"I"}) = [0.2, 0.1, 0.5]$$
$$\mathbf{k}_2(\text{"love"}) = [0.9, 0.7, 0.2]$$
$$\mathbf{k}_3(\text{"cats"}) = [0.4, 0.3, 0.8]$$

Values (representations):

$$\mathbf{v}_1 = [1.0, 0.2, 0.3]$$
$$\mathbf{v}_2 = [0.5, 0.9, 0.1]$$
$$\mathbf{v}_3 = [0.3, 0.4, 0.8]$$

Step 1: Calculate attention scores (dot products)

$$\text{score}_1 = \mathbf{q}^T\mathbf{k}_1 = 0.8(0.2) + 0.6(0.1) + 0.3(0.5) = 0.16 + 0.06 + 0.15 = 0.37$$
$$\text{score}_2 = \mathbf{q}^T\mathbf{k}_2 = 0.8(0.9) + 0.6(0.7) + 0.3(0.2) = 0.72 + 0.42 + 0.06 = 1.20$$
$$\text{score}_3 = \mathbf{q}^T\mathbf{k}_3 = 0.8(0.4) + 0.6(0.3) + 0.3(0.8) = 0.32 + 0.18 + 0.24 = 0.74$$

Step 2: Apply softmax (without scaling for simplicity)

$$\alpha_1 = \frac{\exp(0.37)}{\exp(0.37) + \exp(1.20) + \exp(0.74)} = \frac{1.45}{1.45 + 3.32 + 2.10} = \frac{1.45}{6.87} \approx 0.21$$
$$\alpha_2 = \frac{\exp(1.20)}{6.87} = \frac{3.32}{6.87} \approx 0.48$$
$$\alpha_3 = \frac{\exp(0.74)}{6.87} = \frac{2.10}{6.87} \approx 0.31$$

Check: $0.21 + 0.48 + 0.31 = 1.00$
Step 3: Compute weighted sum

$$\mathbf{c} = \alpha_1\mathbf{v}_1 + \alpha_2\mathbf{v}_2 + \alpha_3\mathbf{v}_3$$
$$= 0.21[1.0, 0.2, 0.3] + 0.48[0.5, 0.9, 0.1] + 0.31[0.3, 0.4, 0.8]$$
$$= [0.21, 0.042, 0.063] + [0.24, 0.432, 0.048] + [0.093, 0.124, 0.248]$$
$$= [0.543, 0.598, 0.359]$$

Interpretation:

- Highest attention (0.48) on "love" itself - makes sense!
- Medium attention (0.31) on "cats" - the object being loved
- Low attention (0.21) on "I" - less relevant for translating "love"

The context vector $\mathbf{c}$ now encodes relevant information from input, weighted by importance.

8.3 Gradient Descent Optimization

Gradient Vector

The **gradient** of a scalar function $f : \mathbb{R}^n \to \mathbb{R}$ is a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

The gradient vector points in the direction of **steepest ascent**.

Gradient Descent Update Rule:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \alpha \nabla f(\mathbf{x}_t)$$

where:

- $\alpha > 0$ is the learning rate (step size)
- $-\nabla f$ points toward steepest descent (minimum)

Key Properties:

- Gradient magnitude $\|\nabla f\|$ indicates steepness
- At minimum: $\nabla f = \mathbf{0}$ (all partial derivatives zero)
- Orthogonal to level curves/surfaces

ML Application 9.3.1: Training Linear Regression

Scenario: Fit line to data points (x_i, y_i) using gradient descent.
Model: $\hat{y} = w_1 x + w_2$ (parameters: $\mathbf{w} = [w_1, w_2]^T$)
Loss function: Mean Squared Error

$$L(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2 = \frac{1}{m}\sum_{i=1}^{m}(y_i - w_1 x_i - w_2)^2$$

Training data:

$$(x_1, y_1) = (1, 3)$$
$$(x_2, y_2) = (2, 5)$$
$$(x_3, y_3) = (3, 7)$$

Step 1: Calculate gradient

$$\frac{\partial L}{\partial w_1} = \frac{-2}{m}\sum_{i=1}^{m} x_i(y_i - w_1 x_i - w_2)$$

$$\frac{\partial L}{\partial w_2} = \frac{-2}{m}\sum_{i=1}^{m}(y_i - w_1 x_i - w_2)$$

Iteration 1: Initialize $\mathbf{w}_0 = [0, 0]$, $\alpha = 0.1$
Calculate predictions:

$$\hat{y}_1 = 0(1) + 0 = 0, \quad \text{error} = 3 - 0 = 3$$
$$\hat{y}_2 = 0(2) + 0 = 0, \quad \text{error} = 5 - 0 = 5$$
$$\hat{y}_3 = 0(3) + 0 = 0, \quad \text{error} = 7 - 0 = 7$$

Gradient:

$$\frac{\partial L}{\partial w_1} = \frac{-2}{3}[1(3) + 2(5) + 3(7)] = \frac{-2}{3}[3 + 10 + 21] = \frac{-68}{3} \approx -22.67$$

$$\frac{\partial L}{\partial w_2} = \frac{-2}{3}[3 + 5 + 7] = \frac{-30}{3} = -10$$

Gradient vector: $\nabla L(\mathbf{w}_0) = [-22.67, -10]^T$
Update:

$$\mathbf{w}_1 = \mathbf{w}_0 - \alpha\nabla L(\mathbf{w}_0) = [0, 0] - 0.1[-22.67, -10]$$
$$= [0, 0] + [2.267, 1.0] = [2.267, 1.0]$$

Iteration 2: With $\mathbf{w}_1 = [2.267, 1.0]$
Predictions:

$$\hat{y}_1 = 2.267(1) + 1.0 = 3.267, \quad \text{error} = -0.267$$
$$\hat{y}_2 = 2.267(2) + 1.0 = 5.534, \quad \text{error} = -0.534$$
$$\hat{y}_3 = 2.267(3) + 1.0 = 7.801, \quad \text{error} = -0.801$$

New gradient:

$$\frac{\partial L}{\partial w_1} = \frac{-2}{3}[1(-0.267) + 2(-0.534) + 3(-0.801)] \approx 2.136$$

$$\frac{\partial L}{\partial w_2} = \frac{-2}{3}[-0.267 - 0.534 - 0.801] \approx 1.068$$

9 Assessment Questions

This section provides comprehensive assessment questions ranging from basic conceptual understanding to advanced applications. Solutions are provided for selected problems.

9.1 Basic Level (11-20)

6. Calculate the angle between vectors $\mathbf{u} = [1, 1]$ and $\mathbf{v} = [1, 0]$.
7. Normalize the vector $\mathbf{w} = [3, 4, 0]$ to unit length.
8. Project vector $\mathbf{b} = [2, 3]$ onto $\mathbf{a} = [1, 0]$.
9. Are vectors $[1, 2, 3]$ and $[2, 4, 6]$ linearly independent? Justify.
10. Find the span of vectors $\{[1, 0], [0, 1], [1, 1]\}$ in $\mathbb{R}^2$.
11. Determine if $W = \{[x, 2x] \mid x \in \mathbb{R}\}$ is a subspace of $\mathbb{R}^2$.
12. Write the equation of a hyperplane through $(1, 1, 1)$ with normal vector $[1, 2, 3]$.
13. Calculate the distance from point $(2, 3)$ to hyperplane $x + y - 4 = 0$.
14. Express $[5, 7]$ as a linear combination of $[1, 2]$ and $[2, 1]$.
15. If $\dim(V) = 5$, what is the maximum number of linearly independent vectors in V?

9.2 Intermediate Level (21-30)

16. Given word embeddings: $\mathbf{v}_{\text{king}} = [0.8, 0.2, 0.6]$, $\mathbf{v}_{\text{man}} = [0.6, 0.1, 0.4]$, $\mathbf{v}_{\text{woman}} = [0.5, 0.2, 0.3]$. Calculate $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}}$ and interpret.
17. In PCA, explain why principal components are orthogonal (use projection concept).
18. Calculate attention weights for query $\mathbf{q} = [1, 0]$ and keys $\mathbf{k}_1 = [1, 1]$, $\mathbf{k}_2 = [0, 1]$.
19. Show that the intersection of two subspaces is also a subspace.
20. Prove that if $\mathbf{v}_1, \mathbf{v}_2$ are orthogonal unit vectors, then $\{\mathbf{v}_1, \mathbf{v}_2\}$ is linearly independent.
21. Given SVM with $\mathbf{w} = [3, 4]$ and $b = -12$, find the margin width.
22. Perform one gradient descent update for $f(x, y) = x^2 + y^2$ starting from $(2, 3)$ with $\alpha = 0.1$.
23. Calculate cosine similarity between document vectors: $\mathbf{d}_1 = [3, 1, 0, 2]$, $\mathbf{d}_2 = [1, 0, 2, 1]$.
24. Explain why multicollinearity in regression relates to linear dependence of feature vectors.
25. In a neural network, if input $\mathbf{x} = [1, 2]^T$, weights $\mathbf{w} = [0.5, -0.3]^T$, bias $b = 0.1$, calculate pre-activation z.

9.3 Advanced Level (31-40)

26. Derive the formula for vector projection using the principle of orthogonality.
27. Prove that the null space of a matrix A is a subspace.
28. Given covariance matrix $C = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$, find eigenvectors (PCA directions).
29. Implement attention mechanism: Given Q, K, V matrices, write the formula for multi-head attention.
30. Explain how the rank of a design matrix X relates to feature independence in regression.
31. Derive the gradient of loss $L = \|\mathbf{y} - X\mathbf{w}\|^2$ with respect to $\mathbf{w}$.

32. In word embeddings, explain why sim("cat","dog") > sim("cat","car") emerges from training.
33. Calculate the volume of parallelepiped spanned by $[1,0,0]$, $[0,2,0]$, $[1,1,3]$ (use determinant).
34. Show that if $\{\mathbf{v}_1,\ldots,\mathbf{v}_k\}$ spans $\mathbb{R}^n$ and $k<n$, the vectors must be linearly dependent.
35. Design a feature engineering pipeline: Given raw features $[x_1,x_2]$, create polynomial features up to degree 2 and explain the resulting vector space dimension.

9.4 Solutions to Selected Questions

Solution 11: Angle Between Vectors

$\mathbf{u}=[1,1]$, $\mathbf{v}=[1,0]$

$$\cos\theta=\frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}=\frac{1(1)+1(0)}{\sqrt{2}\cdot 1}=\frac{1}{\sqrt{2}}$$

$$\theta=\arccos\left(\frac{1}{\sqrt{2}}\right)=45=\frac{\pi}{4}\text{ radians}$$

Solution 12: Normalization

$\mathbf{w}=[3,4,0]$

$$\|\mathbf{w}\|=\sqrt{9+16+0}=5$$

$$\hat{\mathbf{w}}=\frac{1}{5}[3,4,0]=[0.6,0.8,0]$$

Verify: $0.6^2+0.8^2=0.36+0.64=1$

Solution 14: Linear Independence

Vectors $[1,2,3]$ and $[2,4,6]$
Notice: $[2,4,6]=2[1,2,3]$
Since one is a scalar multiple of the other, they are **linearly dependent**.
Alternatively: $2\mathbf{v}_1-1\mathbf{v}_2=\mathbf{0}$ (non-trivial solution exists).

Solution 26: SVM Margin

$\mathbf{w}=[3,4]$, $b=-12$

$$\text{Margin}=\frac{2}{\|\mathbf{w}\|}=\frac{2}{\sqrt{9+16}}=\frac{2}{5}=0.4$$

The perpendicular distance between margin boundaries is 0.4 units.

Solution 27: Gradient Descent

$f(x, y) = x^2 + y^2$, start at $(2, 3)$, $\alpha = 0.1$

Step 1: Calculate gradient

$$\nabla f = \begin{bmatrix} 2x \\ 2y \end{bmatrix} = \begin{bmatrix} 2(2) \\ 2(3) \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

Step 2: Update

$$\begin{bmatrix} x_{\text{new}} \\ y_{\text{new}} \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \end{bmatrix} - 0.1 \begin{bmatrix} 4 \\ 6 \end{bmatrix} = \begin{bmatrix} 1.6 \\ 2.4 \end{bmatrix}$$

New position: $(1.6, 2.4)$ (closer to minimum at origin).

Solution 31: Projection Derivation

Goal: Project $\mathbf{b}$ onto $\mathbf{a}$.

Setup: Decompose $\mathbf{b} = \mathbf{p} + \mathbf{r}$ where:

- $\mathbf{p}$ is parallel to $\mathbf{a}$: $\mathbf{p} = c\mathbf{a}$
- $\mathbf{r}$ is perpendicular to $\mathbf{a}$: $\mathbf{r} \perp \mathbf{a}$

Orthogonality condition: $\mathbf{r} \cdot \mathbf{a} = 0$

$$(\mathbf{b} - c\mathbf{a}) \cdot \mathbf{a} = 0$$

$$\mathbf{b} \cdot \mathbf{a} - c(\mathbf{a} \cdot \mathbf{a}) = 0$$

$$c = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{a} \cdot \mathbf{a}} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|^2}$$

Therefore:

$$\text{proj}_{\mathbf{a}}\mathbf{b} = c\mathbf{a} = \left(\frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|^2} \right) \mathbf{a}$$

10 Summary and Key Takeaways

Essential Concepts

Angles and Projections:

- Cosine similarity measures directional similarity: $\cos\theta = \frac{\mathbf{u}\cdot\mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$
- Unit vectors simplify calculations and focus on direction
- Projections decompose vectors: critical for PCA and optimization

Linear Independence and Basis:

- Independent vectors provide efficient spanning (no redundancy)
- Basis vectors form coordinate systems
- Multicollinearity = dependence = unstable models
- Span determines model capacity and expressiveness

Vector Spaces:

- Subspaces: closed under linear combinations
- Hyperplanes: $(n-1)$-dimensional decision boundaries
- Distance to hyperplane: $\frac{|\mathbf{w}^T\mathbf{x}+b|}{\|\mathbf{w}\|}$

Advanced Applications:

- Embeddings map discrete objects to continuous vectors
- Attention uses dot products to compute relevance
- Gradient descent follows negative gradient to minimize loss
- Vector operations enable all neural computations

ML/DL Connections

Why These Concepts Matter:

1. **Feature Engineering:** Linear combinations create derived features
2. **Dimensionality Reduction:** PCA projects onto principal directions
3. **Classification:** Hyperplanes separate classes (SVM, logistic regression)
4. **Embeddings:** Semantic similarity via vector distance/angle
5. **Attention:** Weighted combinations based on relevance scores
6. **Optimization:** Gradient vectors guide parameter updates
7. **Regularization:** Vector norms control model complexity